Unit 3: Products of Computational Linguistics

21CSE552T · COMPUTATIONAL LINGUISTICS

Unit 3: Products of Computational Linguistics

8 topics · complete unit notes

Classification of Applied Linguistic Systems

Before studying the methods and techniques of computational linguistics, it is worthwhile to survey what the field actually delivers. The discipline exists to serve concrete needs in business and science, and a review of its practical results motivates the methods that follow. This section catalogues the applied linguistic systems that process, understand, and generate human language. It groups them into a small set of broad classes, breaks text preparation into its component tasks, and separates the products that are already deployed from those that remain prospective. Later units explain how individual systems work; the aim here is to build the map first.

Why survey the products first

The authors of the source text deliberately open with outcomes rather than theory, because students learn methods faster once they can see what those methods are for. The unifying object of study is the applied linguistic system.

DEFINITION

An **applied linguistic system** is a software system that performs a practical language task such as editing, searching, translating, or understanding natural-language text.

Applications are read in a very broad sense here, spanning word processing, text processing, dialogue, and language understanding. Some of these tasks already provide satisfactory solutions to the user, especially for English, while other tasks and other languages have been under continuous research for decades. A spell checker, for instance, is satisfying to use in English but far weaker for morphologically rich languages, which is precisely why research continues. Extrapolating current trends could yield entirely new types of system, but that speculation lies outside the scope of the source text.

The map of applied systems

Applied linguistic systems are widely used in business and scientific domains, and they fall into a small set of broad classes. The most important classes are text preparation (text editing), information retrieval, automatic translation, natural language interfaces, factual data extraction, text generation, natural language understanding, and character and speech recognition. Each branch has a familiar example: a word processor, a search engine, a translator, a database queried in plain English, reading a price out of an invoice, producing a report from data, answering a question, and scanning a printed page. The remaining sections zoom into each branch in turn.

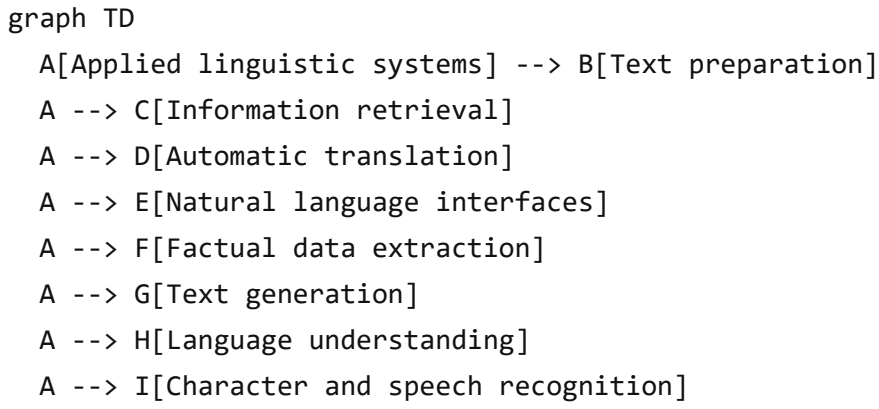


Figure: the main classes of applied linguistic systems.

Text preparation, broadly defined

Text preparation is the most familiar class, because it is built into every word processor. It is not one feature but a family of them, bundled together so completely that users rarely notice the separate systems at work.

DEFINITION

Text preparation, or text editing in a broad sense, is the class of tasks that help an author produce clean, correct, and well-linked text.

The class covers correctness at several levels of language, and it also covers connecting words to their meanings and references. Its subtasks range from mechanical layout, such as deciding where a line may break, to stylistic judgement about awkward wording. Hyphenation, the red and blue underlines of a spell and grammar checker, and cross-references are all separate applied systems that a single program presents as one.

The five text-preparation tasks

The five subtasks are best learned in order of increasing linguistic difficulty. Automatic hyphenation decides where a word may break at the end of a line. Spell checking detects typographic and spelling errors, such as flagging "teh". Grammar checking detects grammatical errors, such as the disagreement in "he go". Style checking detects stylistic errors and awkward or overly wordy phrasing. Referencing links specific words and word combinations to each other through their semantic connections, for example tying a pronoun back to the noun it refers to.

Task	What it detects or does
Automatic hyphenation	Where a word may break at a line end
Spell checking	Typographic and spelling errors
Grammar checking	Grammatical errors, such as disagreement
Style checking	Stylistic errors and awkward wording
Referencing	Words, word combinations, and semantic links between them

EXAM POINT

Learn these five subtasks in order. Hyphenation is the most mechanical; referencing is the most linguistically demanding, because it depends on meaning. Difficulty rises down the column, which is why spell checkers are mature but style checkers are not.

Information retrieval

The next class turns from producing text to finding it. Information retrieval operates over scientific, technical, and business document databases, and its everyday example is a web search engine, although the original setting was in-house archives.

DEFINITION

Information retrieval is finding the documents in a collection that are relevant to a user's query.

The goal is relevance ranking rather than exact matching: the system returns a ranked list of documents, not a single answer. Modern retrieval can also supply passages to a language model, though the core task remains the same as it always was.

```
graph LR
  A[User query] --> B[Search over document base]
  B --> C[Ranked relevant documents]
```

Figure: a query is matched against a document base to return ranked results.

Translation and interfaces

Two further classes both involve moving between languages, but in different directions. Automatic translation converts text from one natural language to another. It is a long-standing goal of the field and is now dominated by neural systems. Natural language interfaces let users query databases and other systems in plain language, with the system mapping a sentence to a formal query and thereby removing the need to learn a query language.

Automatic translation

- Converts text from one natural language to another.
- A long-standing goal of the field.
- Now dominated by neural systems.

Natural language interfaces

- Let users query databases in plain language.
- The system maps a sentence to a formal query.
- Removes the need to learn a query language.

KEY POINT

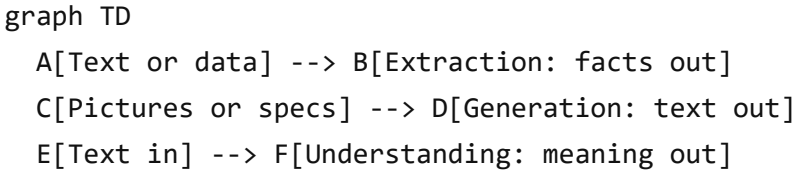
Translation moves between two natural languages; a natural language interface moves from a natural language into a formal one that a machine can execute.

For example, a natural language interface lets a user type "which orders shipped late last month" and generates the database query automatically. Large language models now blur the line between these two tasks, but the underlying tasks remain distinct: one produces text in another human language, the other produces an executable formal query.

Extraction, generation, and understanding

Three classes sit at the deeper, meaning-oriented end of the field. Factual data extraction pulls specific facts from business or scientific texts, for instance extracting a company name and an amount from a contract. Text generation produces text from pictures or formal specifications, such as generating a weather report from numeric data. Natural language understanding aims at grasping the meaning of text, for example answering a comprehension question about a paragraph. All three depend on the referencing and semantic links introduced under text preparation, and this is the area where modern large language models are strongest.

- **Factual data extraction** pulls specific facts from business or scientific texts.
- **Text generation** produces text from pictures or formal specifications.
- **Natural language understanding** aims at grasping the meaning of text.
- These three sit at the deeper, meaning-oriented end of the field.



Recognition at the boundary

Two final classes sit at the input boundary of the field. They do not analyse meaning; they turn a raw signal into text that everything else can consume, so their output feeds every other class listed above.

DEFINITION

Optical character recognition converts an image of printed or written text into machine-readable characters, while **speech recognition** converts spoken audio into text.

A typical pipeline scans a printed invoice with optical character recognition and then runs factual extraction on the resulting text. Speech recognition now feeds voice assistants directly into understanding and dialogue systems. In both cases the recogniser is a gateway that produces the text the rest of the field then processes.

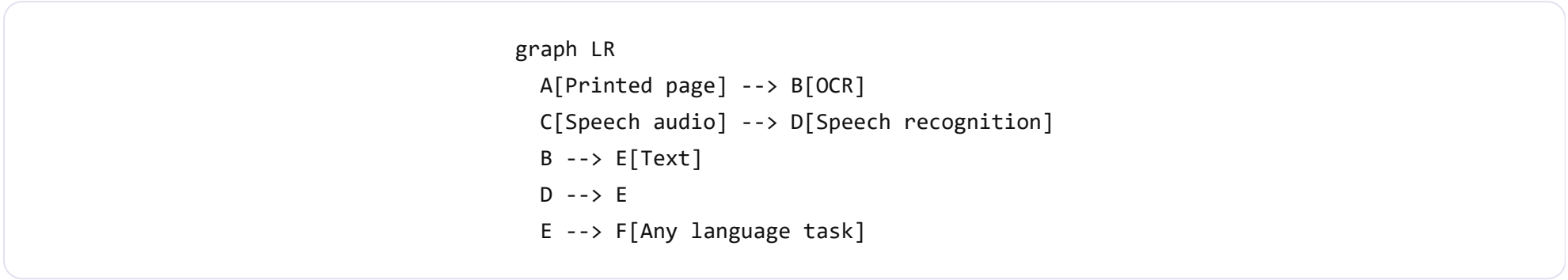


Figure: recognition converts raw signals into text that any language task can use.

Present versus prospective products

The classes above are not equally mature. Some are present products, deployed and in daily use, while others are prospective, still under research or merely extrapolated from current trends. This course studies the mature products in detail and only sketches the prospective ones.

Aspect	Present products	Prospective products
Status	Deployed and in daily use	Under research or extrapolated
Typical maturity	Strong for English	Weaker or open for many languages
Examples	Spell checking, search, translation	Full understanding, robust generation
Scope in this course	Studied in detail	Sketched only, not the focus

Extrapolating current trends may yield entirely new system types, but that speculation lies outside the scope of the source text. A spell checker is a present product that can be opened now, whereas genuine open-domain understanding is still maturing. Not every language task is solved; language remains hard.

Summary

Computational linguistics is reviewed by its products before its methods, because a survey of results motivates the methods that follow. Applied linguistic systems serve business and scientific needs in a broad sense, and they group into a small set of classes: text preparation, information retrieval, automatic translation, natural language interfaces, factual data extraction, text generation, natural language understanding, and character and speech recognition. Text preparation itself splits into automatic hyphenation, spell checking, grammar checking, and style checking, plus referencing, in order of rising linguistic difficulty. Some of these products are mature today, especially for English, while others remain prospective and heavily language-dependent. The next topic drills into the first and most mechanical text-preparation task, automatic hyphenation, and the spell checking that builds on it.

Automatic Hyphenation and Spell Checking

The first two subtasks of text preparation are the most familiar and the most mature: automatic hyphenation, which decides where a word may break at the end of a line, and spell checking, which detects and corrects errors in individual words. Both are built into every modern word processor, yet they differ sharply in the amount of linguistic knowledge they require. Hyphenation needs only a little information about letters and syllables, while spell checking needs a full list of valid words and a model of how errors arise. This section defines each task, sets out the linguistic knowledge each one uses, works through concrete examples in Spanish and English, and marks the limits that spell checkers still have not crossed. Throughout, keep the earlier ordering in mind: hyphenation is the most mechanical text-preparation task, and spell checking sits just above it.

Automatic hyphenation

DEFINITION

Automatic hyphenation is the task of finding the correct positions at which a word may be split when it does not fit within the accepted margins at the end of a line.

When a word occurring at the end of a line is too long to fit within the accepted margins, part of it is moved to the next line. The word is thus wrapped: split and partially transferred, with a hyphen marking the break. The purpose is presentational. Hyphenation improves the outer appearance of computer-produced texts by adjusting their right margins, it saves paper, and it preserves the impression of smooth reading, much as an unhyphenated page would. The majority of well-known text editors now supply a hyphenation tool; Microsoft Word, for example, offers it as a menu item.

Where a word may be split

A word cannot be broken at an arbitrary point. Wrapping is allowed only at specific positions, which generally, though not always, coincide with syllable boundaries.

KEY POINT

The valid break points are usually syllable boundaries, but not every syllable boundary is a legal break, and some legal breaks depend on facts beyond syllable structure.

Consider English. The word forms below may be split at the marked positions: teach-er, moth-er, wash-ing, graph-ic, beau-ti-ful. The same words may not be split at other positions, and the following breaks are all wrong: teac-her, mot-her, was-hing, grap-hic. The wrong cases show that certain letter combinations must never be separated. The consonant digraphs ch, th, sh, and ph each spell a single sound and behave as single units, so teac-her and grap-hic are illegal because they cut through ch and ph. Vowel pairs that spell one sound, such as the ea in beau-ti-ful or the ai in main-tain, also stay together.

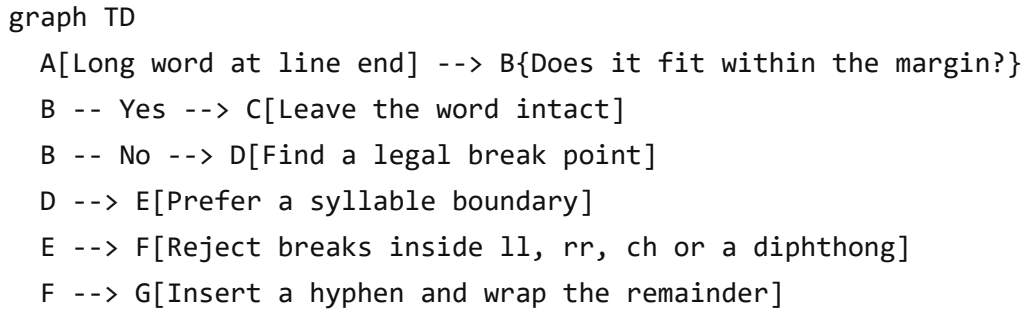


Figure: the decision procedure for wrapping a word at the end of a line.

The linguistic knowledge hyphenation uses

The linguistic information required is usually rather limited. A program needs to know which letters are vowels, which are consonants, and which letter combinations are inseparable.

DEFINITION

An **inseparable letter combination** is a group of letters, such as the Spanish consonant pairs ll, rr, and ch or the diphthongs io, ue, and ai, that must not be divided by a hyphen.

For Spanish the vowels are a, e, i, o, and u, and the consonants are b, c, d, f, g, and the remaining letters. With the vowels, the consonants, and the inseparable combinations identified, a fairly simple program can hyphenate ordinary Spanish text correctly.

When simple rules are not enough

The best quality of hyphenation can demand more detailed information about each word. The correct break sometimes depends on the morphemic structure of the word, that is, on how the word is built from meaningful parts. Compare mis-hap with fish-ing. Both contain the letters s and h side by side, but in fishing the sh is a digraph spelling one sound and must stay together, while in mishap the s ends the prefix mis and the h begins hap, so the two letters belong to different morphemes and must be separated. The letters alone cannot decide this; only knowledge of how the word is built can. The same applies to un-ionized, meaning not ionized, against union-ized, formed from union, where identical letters split in two different places depending on the morphemes. Only a dictionary-based program, one that stores each word together with its structure, can take all such considerations into account.

DEFINITION

A **dictionary-based hyphenation program** stores individual words together with their permitted break points and, where needed, their morphemic structure, rather than relying on general letter rules alone.

For English, only dictionary-based programs give perfect results, because English spelling relates to pronunciation and word structure in irregular ways. For Spanish, rather simple rule-based programs are usually sufficient, provided one is willing to tolerate occasional mistakes on error-prone foreign words such as Shiller.

EXAM POINT

Hyphenation for Spanish can be done with a simple rule-based program using vowels, consonants, and inseparable combinations; English needs a dictionary-based program for perfect results. Be ready to justify the difference in terms of morphemic structure and word origin.

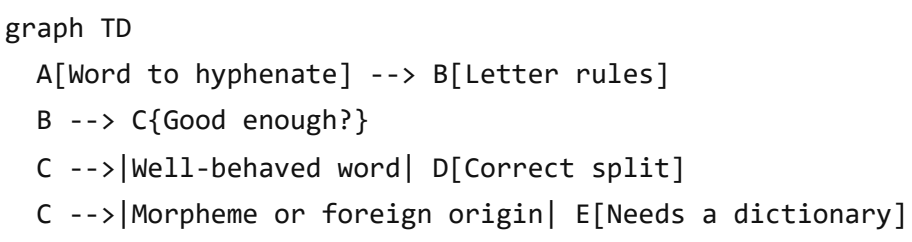


Figure: letter rules handle most words, but morphologically complex or foreign words need a dictionary.

Spell checking

DEFINITION

Spell checking is the detection and correction of typographic and orthographic errors in a text, at the level of the individual word occurrence considered out of its context.

The defining phrase is "out of its context." A spell checker looks at each word on its own and asks whether that string could be a valid word of the language, without considering the surrounding words. This is what separates it from grammar checking, which does look at context.

Two kinds of error

Nobody writes without error. The mistakes a spell checker addresses fall into two kinds.

DEFINITION

A **typographic error**, or typo, is an accidental slip in typing, such as pressing a wrong key adjacent to the intended one, omitting a letter, or mistiming the movements of the fingers.

DEFINITION

A **spelling error** is a mistake made because the writer does not know the correct spelling of a word, which happens most often in a foreign language.

Both kinds usually produce a string that is not a possible word of the language. Examples of such impossible strings are the forms groop, greit, misanderstand, systema, and seperate. In each case an initial asterisk is conventionally used to mark a string that contains an error.

Aspect	Typographic error	Spelling error
Cause	Accidental slip while typing	Not knowing the correct spelling
Typical writer	Anyone, even an expert	Often a non-native speaker
Example	greit for great	seperate for separate
Detected the same way	Yes	Yes

What a spell checker does

A spell checker works in two stages, detection and then correction.

First, the program merely detects strings that are not correct words of the language. The working assumption is that most orthographic and typographic errors produce strings that are impossible as separate words. When such an impossible string is found, the program highlights it and stops, so the user can fix it, either by hand or with the program's help. Inserting groop, greit, misanderstand, or systema into a text causes the

checker to halt at the offending string.

```
graph TD
  A[Word occurrence] --> B{Is it in the word list?}
  B -- Yes --> C[Accept, move on]
  B -- No --> D[Highlight as an error]
  D --> E[Propose similar valid words]
  E --> F[User chooses a correction]
```

Figure: detection followed by candidate proposal in a spell checker.

Second, the functions of a spell checker can be more versatile than detection alone. The program can propose a set of existing words that are similar enough, in some defined sense, to the corrupted string, and the user can choose one as the correct version without retyping. For a short corrupted string such as greit, a checker offers several candidates, among them great, grit, and grey, because many real words sit close to it. For long strings the checker often offers only one or two candidates, or none: for systema it offers only the correct word system.

DEFINITION

An **orthographic corrector** is a program that both detects incorrect strings and proposes corrections; in English such programs are usually called spell checkers.

Spell checkers are considered very helpful and are used by millions of people worldwide. Most modern text editors ship with integrated spell checkers, and a program such as Microsoft Word uses a separate checker for each natural language present in the text.

The linguistic knowledge spell checking uses

The amount of linguistic information a spell checker needs is much greater than that needed for hyphenation. A simple but resource-consuming approach keeps a list, or dictionary, of every valid word of the language. Beyond that list, the program needs a criterion of similarity between words, so that it can propose plausible corrections, and some assumptions about which typographic and spelling errors are most common. A deeper treatment of the correction problem requires detailed knowledge of morphology, because morphology lets the program store word roots and rules for their variation rather than every inflected form, which keeps the dictionary to a manageable size.

KEY POINT

Hyphenation needs only letters and inseparable combinations; spell checking needs a full word list, a similarity measure, a model of likely errors, and, for compactness, a knowledge of morphology.

Feature	Automatic hyphenation	Spell checking
Purpose	Split words correctly at line ends	Detect and correct word-level errors
Uses context?	No	No
Core knowledge	Vowels, consonants, inseparable combinations	Full dictionary of valid words
Extra knowledge for quality	Morphemic structure, word origin	Similarity measure, error model, morphology
Amount of knowledge	Limited	Much greater
Simple program adequate?	Yes for Spanish	Only as a resource-heavy word list

What spell checkers still cannot do

Spell checkers have two well-known limits. The first is that detecting an error which accidentally turns one valid word into another valid word is a much harder task, one that needs far more powerful, context-aware tools. The slips then for than, form for from, and their for there all produce real words, so a checker that judges each word out of context cannot notice them.

The second limit concerns correction quality. Although spell checkers have been available for more than twenty years, some obvious correction tasks remain unsolved even for isolated words. For the ungrammatical string teached, the checkers the authors tried did not suggest the correct form taught. The same happens with goed for went and childs for children. These cases involve irregular morphology, where the correct form differs too much from the error for a simple similarity measure to reach it.

EXAM POINT

A spell checker judges each word out of context, so it cannot catch a real-word error such as then for than or form for from. It can also miss corrections that require irregular morphology, such as teached to taught. Both limits follow directly from the "out of context" definition and from a shallow error model.

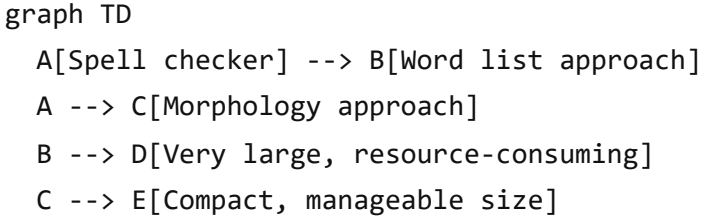


Figure: the two ways to build a spell checker, and the cost of each.

Summary

Automatic hyphenation and spell checking are the first two subtasks of text preparation, and both operate on words considered out of context. Hyphenation splits a word that does not fit at the end of a line, breaking it only at legal positions, which are usually syllable boundaries but never inside inseparable combinations such as the Spanish ll, rr, ch or the diphthongs io, ue, ai. It needs little linguistic knowledge, so a simple rule-based program suffices for Spanish, whereas English needs a dictionary-based program because correct breaks depend on morphemic structure and word origin. Spell checking detects and corrects typographic errors, which are accidental slips, and spelling errors, which come from not knowing the correct form. It first flags strings that cannot be valid words, then proposes similar valid words as candidates; a program that does both is an orthographic corrector. It requires far more knowledge than hyphenation, namely a full word list, a similarity measure, an error model, and, for a compact dictionary, a command of morphology. Its two enduring limits, missing real-word errors and failing on irregular forms such as teached to taught, both follow from judging each word out of its context.

Grammar Checking and Style Checking

Grammar checking and style checking are the third and fourth subtasks of text preparation, sitting one step beyond spell checking in linguistic difficulty. A spell checker inspects one word at a time and asks only whether that string exists in the language. A grammar checker must look across adjacent words, or even the whole sentence, to judge whether the words fit together correctly. A style checker goes further still, judging not whether the text is correct but whether it is well written for its purpose. Both tasks depend on knowledge that a mere word list cannot supply: grammatical categories, syntactic rules, and, ultimately, full parsing. This section defines both tasks, works through their characteristic error types with concrete examples, explains why simplistic techniques fail, and closes with the practical rule that the author, not the program, holds the final say.

Learning outcomes

By the end of this topic you should be able to state precise definitions of grammar checking and style checking, describe the kinds of error each one targets, give concrete examples of grammatical and stylistic errors, explain why only full syntactic parsing can solve grammar checking properly, understand why narrow-context shortcuts produce both missed errors and false alarms, and describe the simple statistical measures that commercial style checkers actually use.

What grammar checking is

DEFINITION

Grammar checking is the detection and correction of grammatical errors in a text by taking into account adjacent words, or the whole sentence, rather than each word in isolation.

This is a much harder task than checking orthography. A spell checker can treat every word independently, but grammatical correctness is a property of how words combine. A grammar error is one that violates the syntactic laws of a language or the laws governing the structure of a sentence. Because the words that must fit together can sit far apart, the checker cannot rely on looking at a single word, and often cannot even rely on looking at a small window of neighbouring words.

KEY POINT

Spell checking asks whether a word exists. Grammar checking asks whether the words, each of which may exist, are correctly combined.

Grammatical errors: agreement

The clearest class of grammatical error is a failure of agreement. In English, a determiner and its noun must agree in number. In the combination *this books, each word exists on its own, yet together they form a syntactically ill-formed phrase, because the singular determiner does not agree with the plural noun. A second kind of agreement links the subject with the main verb in number and person, as in the ill-formed *she have, where the third-person singular subject does not match the verb form. Languages that mark gender on adjectives, such as Spanish or Hindi, add a third agreement law of exactly the same kind, so the problem below is not peculiar to English.

DEFINITION

Agreement is the requirement that related words in a sentence share grammatical features such as gender, number, and person.

The difficulty is that the words which must agree can be located in quite different parts of the sentence. In *The list of the new students are ready, the verb are must agree with the singular subject list, but the two are separated by the intervening phrase of the new students, so a program cannot find the error by looking only at neighbouring words. This example is worse than it first appears: the word immediately before are is the plural students, which agrees with it perfectly, so a checker that inspects only the adjacent pair sees nothing wrong at all.

Two agreement laws in English, both easy for humans and hard for programs:

- Determiner with noun, in number.
- Subject with main verb, in number and person.

- The agreeing words may be far apart in the sentence.

```mermaid
graph LR
  A["The list"] -->|must agree| B["are ready"]
  A --> C["of the new students"]
  C --> B
```

```mermaid
graph TD
  A["this: singular"] --> C{Agree?}
  B["books: plural"] --> C
  C -->|no match| D[Agreement error]
```

*Figure: the feature clash behind this books. Each word is real; only the combination fails.

Other grammatical errors

Agreement is not the only kind of grammatical error. Prepositions are a frequent source of mistakes, as in *he is good football (a missing preposition, where *at* is needed), *she discussed about the problem (an unwanted one, since *discuss* takes a bare object), or *he is married with Mary (the wrong one, where *to* is correct). Some of these syntactic errors are subtle enough that even a native speaker may not notice them at once.

Error type	Example (ill-formed)	What is wrong
Determiner to noun agreement	*this books	Number mismatch
Subject to verb agreement	*she have	Person and number mismatch
Distant agreement	*The list of the new students are ready	Verb disagrees with a distant subject
Missing preposition	*he is good football	*at* is required
Unwanted preposition	*she discussed about the problem	*discuss* takes a bare object
Incorrect preposition	*he is married with Mary	*to* is the correct choice

EXAM POINT

Be ready to classify a grammatical error by type and to explain why distant agreement, such as noun and adjective separated by an intervening phrase, cannot be caught by inspecting only adjacent words.

Why grammar checking needs parsing

It became clear long ago that only a complete syntactic analysis, that is, full parsing of the text, can provide an acceptable solution to grammar checking. To decide whether are agrees with the correct noun, the program must first know which noun is the subject of the verb, and that requires building the sentence structure.

DEFINITION

Parsing is the automatic construction of the syntactic structure of a sentence, identifying how its words group and relate.

Because such parsing is difficult, commercial grammar checkers have historically been rather primitive and rarely gave users useful help. A well-known computer journal, Windows Sources, remarked in May 1995 that Grammatik, the grammar checker in the WordPerfect editor and perhaps the best in the world at the time, was so imperfect and disorienting that nobody needs a program that is wrong more often than it is right. In later years grammar checkers improved considerably; the one built into Microsoft Word became helpful, though still far from perfect. Modern parsers and language models push this further, but the underlying requirement remains: reliable grammar checking rests on understanding sentence structure.

```
graph TD
  A[Input sentence] --> B[Full syntactic parse]
  B --> C[Recover which words must agree]
  C --> D{Do features match?}
  D -->|no| E[Report grammatical error]
  D -->|yes| F[Accept]
```

Simple checks that sometimes help

Some very frequent errors can be caught with simple operations, without full parsing. Two classes illustrate this.

Article to noun disagreement

- Example: *this books.
- Detectable within two adjacent words.
- Needs the grammatical category of each word.

Real-word errors

- Example: *their* for *there*, *form* for *from*.

- One real word is typed in place of another.
- A plain spell checker misses it, since both words exist.

The accent case is instructive: a spell checker taking words out of context cannot catch it, because omitting the accent turns one existing word into another existing word, namely a personal form of a verb. It is possible, though, to define properties of the immediate context, such as the presence of an agreed article, adjective, or pronoun, that never occur before the verb, and so to infer that the noun reading was intended.

```
graph TD
  A[Simple local check] --> B["Catches this books"]
  A --> C[Misses long-distance clashes]
  A --> D[Raises false alarms]
```

Figure: a narrow local check succeeds in one direction and fails in two.

Why the shortcuts fail

These simplistic techniques fail in too many cases. In *The list of the new students are ready, the disagreement is between list and are, but the nearest context of are is students, so a narrow-window check cannot see the true error.

Worse, a program based on such a shortcut gives too many false alarms. Correct the sentence to The list of the new students is ready and a narrow-context program would now wrongly report a disagreement between the adjacent students and is, because those two words differ in number, even though is correctly agrees with the distant list. One sentence therefore defeats the local check in both directions at once: silent on the real error, noisy on the correct version.

Narrow-context grammar checking produces two failures at once: it misses real errors that span a distance, and it raises false alarms on correct text where adjacent words merely appear to clash.

The final decision rests with the author

Since the author is the only person who definitely knows what he or she meant to write, the final decision must always be left to the user, whether to accept a suggested correction or to leave the text unchanged. A grammar checker advises; it does not overrule.

KEY POINT

A grammar checker is an assistant, not an authority. It flags and suggests, but the author confirms.

What style checking is

DEFINITION

Style checking is the detection of stylistic errors, that is, violations of the correct use of words and word combinations in a language, either in general or within a particular literary genre.

This task is the closest of all text-preparation applications to normative grammars and manuals of style in printed, human-oriented form. A style checker therefore plays a didactic and prescriptive role, guiding authors toward better writing rather than merely toward correct writing.

KEY POINT

Grammar checking targets what is incorrect. Style checking targets what is correct but inappropriate or awkward for the chosen genre.

Kinds of stylistic problems

Style rules are about appropriateness. You are advised not to use vulgar words or purely colloquial constructions in official documents. In formal writing, a sentence should not contain, say, ten instances of the preposition of, and should not run longer than about twenty lines. On the lexical side, it is not recommended to reach for a heavy word where a plain one exists, writing utilise, commence, and terminate in place of use, begin, and end.

Two further examples show subtler problems. In The collection of data in real time is collected by a server, two anomalies appear together: the passive is collected usually reads worse than the active a server collects, and placing words with the same stem so close together, collection and collected, is unwanted. In The recording, reproduction, and simulation of data are functions in all information manipulation systems, the crowd of words ending in the suffix -ion oversteps the bounds of good style.

Stylistic problem	Example	Preferred or reason
Register too informal	vulgar or colloquial words in official text	Use formal vocabulary
Overused preposition	many instances of de in one sentence	Reduce repetition
Over-long sentence	a sentence longer than about twenty lines	Split the sentence
Heavy word for a plain one	utilise, commence, terminate	use, begin, end
Passive where active reads better	the data is collected by a server	a server collects the data
Same stem nearby	collection and collected together	Vary the wording
Suffix overused	many words ending in -ion	Vary the suffixes

What a good style checker would need

To do this well, a style checker should draw on a dictionary of words annotated with usage marks, synonyms, information on the proper use of prepositions, and compatibility with other words. It should also use automatic parsing to detect improper syntactic constructions. Style checkers of this depth exist for English and a few other major languages, but mainly in laboratory versions; commercial style checkers remain rather primitive in what they do.

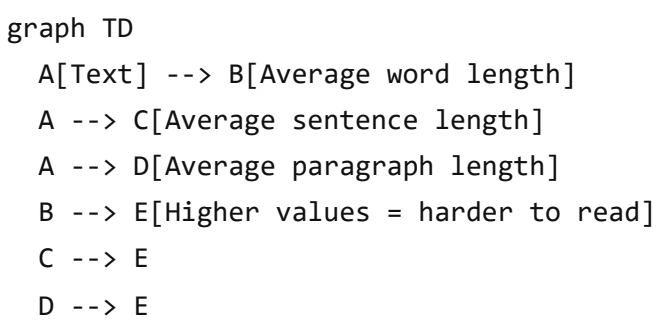
DEFINITION

A usage mark is a label attached to a dictionary word that records its stylistic register or restrictions, such as colloquial, vulgar, or formal.

What commercial style checkers actually do

In practice, many commercial style checkers assess style in a very shallow way, by computing simple statistics.
Typical measures counted by a commercial style checker:

- Average word length, in letters.
- Sentence length, in words.
- Paragraph length, in words and sentences.
- Counts of specific items, such as of or nouns ending in -ion.



The underlying assumption is that the larger the average length of a word, sentence, or paragraph, the harder the text is to read. It is also easy to count occurrences of the preposition of, or of nouns ending in -ion. Such tools can tell a user only that a text is too complicated or awkward for the chosen genre; they usually cannot say how to improve it. Assessing the deeper and more interesting stylistic properties, those tied to lexicon and

syntactic construction, is still regarded as a task for the future.

Deep stylistic assessment, connected with word choice and sentence construction, remains an open research problem rather than a solved product.

Grammar checking versus style checking

Aspect	Grammar checking	Style checking
Judges	Correctness	Appropriateness and quality
Basis	Syntactic laws, sentence structure	Norms of good usage, genre
Knowledge needed	Grammatical categories, full parsing	Usage-marked dictionary, synonyms, parsing
Typical example	*she have, *The list ... are ready	passive is collected, overused -ion
Commercial maturity	Improving but imperfect	Mostly primitive, statistics only

EXAM POINT

Distinguish the two tasks precisely: grammar checking asks whether words are combined lawfully, while style checking asks whether correct words are used well for the genre. Both benefit from parsing; neither is fully solved commercially.

Summary

Grammar checking detects errors in how words combine, such as broken agreement between a noun and its adjective or between a subject and its verb, and such errors can span a distance within the sentence. Only full syntactic parsing solves the task reliably, which is why commercial grammar checkers were long primitive and remain imperfect. Simple two-word checks catch some frequent errors, such as article to noun disagreement or a missing written accent, but they both miss distant errors and raise false alarms on correct text, so the final decision must rest with the author. Style checking is a separate, harder task that judges not correctness but appropriateness for the genre, covering register, repetition, sentence length, foreign words, and overused suffixes. A capable style checker would need a usage-marked dictionary, synonyms, preposition information, and parsing, yet most commercial tools only compute shallow statistics such as average word, sentence, and paragraph length, leaving deep stylistic assessment as a task for the future.

References to Words and Information Retrieval

Two everyday products of computational linguistics sit close to a working author and a working reader. The first is a reference to words, a dictionary that shows which words combine with which, so that a writer can produce text that is correct, flexible, and idiomatic. The second is an information retrieval system, which locates relevant documents inside a very large collection. This topic treats both. By the end you should be able to explain what references to words and word combinations are for and describe CrossLexica as a dictionary of word combinations, define a collocation and say why knowing common combinations is part of linguistic competence, describe how an information retrieval system indexes documents and matches them against queries, and define precision and recall and explain why they trade off against each other.

References to words and word combinations

DEFINITION

A **reference to a word** gives access to the set of words semantically related to it, or to the words that can combine with it in a text.

The purpose of such a reference is to help the author of a text create more correct, flexible, and idiomatic text. This matters because only an insignificant part of all thinkable word combinations is actually permitted in a language. Knowledge of the permitted and common combinations is therefore an important part of the linguistic competence of any author. A native speaker carries that knowledge implicitly, while a dictionary of word combinations makes it explicit and available to a non-native author or to a writing tool.

Two kinds of linguistic tool

Modern reference to words is provided by tools of two kinds: autonomous on-line dictionaries and built-in dictionaries of synonyms. Inside typical text processors, the synonymy dictionaries are usually called thesauri, although the name fits poorly. A genuine thesaurus holds much more than synonyms, including references to generic words, that is, names of superclasses, and to specific words, that is, names of subclasses. The caution to keep in mind is that many products label a plain synonym list as a thesaurus, whereas a real thesaurus records hierarchy, not just synonymy.

```
graph TD
  A[Reference to a word] --> B[Autonomous on-line dictionary]
  A --> C[Built-in synonymy dictionary]
  C --> D[Often called a thesaurus]
  D --> E[Genuine thesaurus adds generic and specific words]
```

Collocations

DEFINITION

A **collocation** is a combination of words that is commonly and idiomatically used together in a language, rather than one assembled word by word.

The classic motivation is a second-language author who needs to know which words legitimately go together. A learner of English may want every verb commonly used with the noun *attention*, such as *pay*, *draw*, or *attract*, or every verb used with *decision*, such as *make*, *reach*, or *take*. Knowing these combinations avoids wrong ones like *do a decision*, a word-by-word translation from a language where the verb for *make* and *do* is the same. Nothing in the meaning of *do* rules it out; it is simply not the verb English pairs with *decision*.

KEY POINT

Correct combinations are language-dependent and cannot be built by translating word by word. A special dictionary is needed for each language.

CrossLexica: a dictionary of word combinations

CrossLexica is a special language-dependent dictionary of word combinations. For a headword it shows related material such as synonyms, attributes, and the verbs that go with it. The figure shows the entry for one headword. Read the figure as follows: for a single word the tool lists the words that can legitimately combine with it, which is exactly the linguistic competence described earlier, now made searchable.

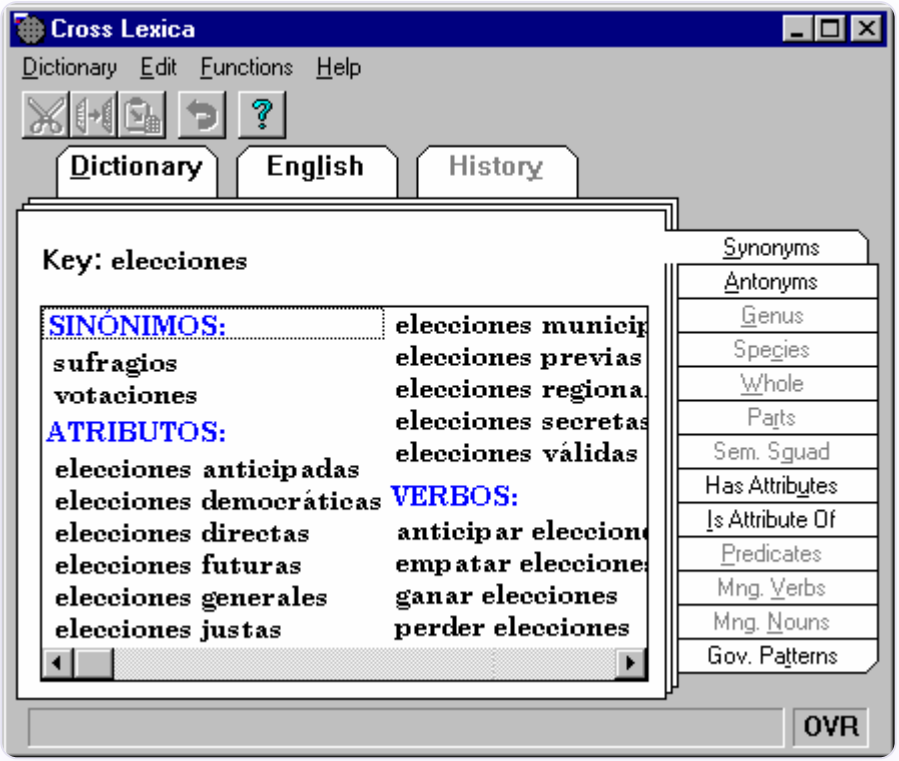


Figure: Bolshakov & Gelbukh, Computational Linguistics (2004)

What such systems do internally

Even a simple lookup hides morphology and syntax. The user types a word, possibly in an inflected form. The system reduces the entered word to its dictionary form, then searches the linguistic database for the relevant words, and finally displays the results in a form convenient for a non-linguist user. These operations are versatile and involve both morphologic issues, namely reducing forms, and syntactic issues, namely which words combine.

```
graph LR
  A[User enters a word] --> B[Reduce to dictionary form]
  B --> C[Search linguistic database]
  C --> D[Display results to non-linguist]
```

Richer semantic resources

DEFINITION

EuroWordNet is a large lexical resource, built on the English dictionary WordNet, that records diverse semantic links between the lexemes of several European languages.

Where a plain synonym list is flat, WordNet organises words into a network of relations. Words are grouped into synonymy groups called **synsets**. Relations between synsets include antonymy, the reference to an opposite meaning, hyponymy, the references to subclasses, hyperonymy, the reference to a superclass, meronymy, the references to parts, and holonymy, the reference to the whole. Semantic links are established even between synsets of different parts of speech. The noun hierarchy is especially deep, averaging 6 to 7 levels and sometimes reaching 15, and its upper levels form an ontology, a presupposed scheme of human knowledge. EuroWordNet transports this hierarchy to other European languages, in particular Spanish, and the effort was tremendous: roughly 25,000 words were elaborated across several languages.

Information retrieval

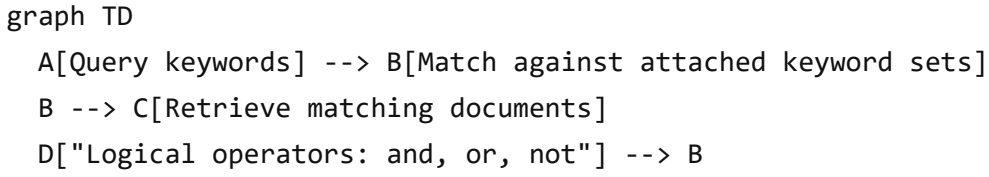
DEFINITION

An **information retrieval system (IRS)** is designed to search for relevant information in large documentary databases.

The point of scale is that an IRS works over a large collection, not a single document. The information sought can be of various kinds, so queries range from "find all documents containing the word *conjugate*" to "find information on the conjugation of irregular verbs". Because these needs differ so widely, from an exact word match to an open topical question, different systems use different methods of search. The earliest systems were built to find scientific articles on a specific topic.

Keywords and queries

Authors usually supply each paper with a set of **keywords**, the terms they consider most important and relevant for the topic. For an article "On means of expressing unreal conditions" the keyword set might be *grammar*, *verb*, *subjunctive*. These sets are attached to the document in the bibliographic database of the IRS, kept either together with the documents or separately. In the simplest case a query names one or more of these keywords as the retrieval condition. In a more elaborate system a query is a longer logical expression using the operators *and*, *or*, *not*.



KEY POINT

Example query: "find the documents on (*nouns* or *adjectives*) and (not *English*)".

Ranking and the "m of n" threshold

A simple but powerful approach popular for non-professional users keeps the query as a set of words. The system first tries to find documents containing all of the words, then all but one, and so on down to those containing only one of the words. The set of keywords is thus considered in a step-by-step transition from conjunction to disjunction of their occurrences. Results are ordered by degree of relevance, which is measured by how many of the keywords are found in the document, so documents with more keywords are presented first. Some systems let the user set a threshold manually, searching for "at least m of n" keywords.

```
graph TD
  A["Find documents with all n keywords"] --> B["Then n minus 1 keywords"]
  B --> C["Down to only 1 keyword"]
  C --> D[Order results by number of keywords matched]
```

EXAM POINT

With a threshold of "at least m of n" keywords: m = n retrieves too few documents, often missing relevant ones; m = 1 retrieves too many because of a high rate of false alarms. A proper balance is needed.

Precision and recall

Recall and precision are usually considered the two main characteristics of an information retrieval system.

DEFINITION

Recall is the number of relevant documents found divided by the total number of relevant documents in the database.

DEFINITION

Precision is the number of relevant documents found divided by the total number of documents found.

Measure	Numerator	Denominator
Recall	Relevant documents found	All relevant documents in database
Precision	Relevant documents found	All documents found

One useful reading is that recall asks "of everything I should have found, how much did I find?" while precision asks "of everything I found, how much was right?".

KEY POINT

These two are contradictory in general: raising one tends to lower the other, so a proper balance must be kept.

The same tension appears in the "m of n" threshold: pushing the threshold up toward m = n favours precision at the cost of recall, while pushing it down toward m = 1 favours recall at the cost of precision.

Automatic indexing and thesaurus enrichment

A specialised IRS usually has an automated indexing subsystem that runs before searches are executed. Given a set of keywords, it adds, using the *or* operator, other related keywords drawn from a hierarchical system of scientific, technical, or business terms. This hierarchy of terms is the **thesaurus** of the IRS and may be an integral part of it. The IRS first enriches the keywords in the query, then compares the enriched query with the previously enriched keyword sets attached to each document, using the criteria described above. Enrichment widens the net using domain knowledge, so after enrichment the average recall of the system usually increases.

```
graph LR
  A[Original query keywords] --> B[Add related terms from thesaurus]
  B --> C[Compare with enriched document keywords]
  C --> D[Retrieve documents]
```

For the query "conjugation" the system could add "morphology", so an article indexed under morphology is still found. After enrichment, average recall usually increases.

Limits of current systems

More recent systems can build keyword sets automatically from the full text of a document, so authors need not supply the keywords themselves, and some modern Internet search engines rest on this idea.

DEFINITION

Automatic abstracting is the automatic extraction of a keyword set from the full text of a document, without the author supplying keywords.

Purely statistical extraction is hard. The most frequent words in business, scientific, or technical texts are purely auxiliary, like prepositions and auxiliary verbs, and they do not reflect the essence of the text. The border between auxiliary and meaningful words cannot be strictly defined. Furthermore, very general term-forming words such as *system* or *device* are too broad to help retrieval, so they are not useful for abstracts. Developers now try to use word combinations and phrases as well as more complicated strategies, but the barrier is deeper.

EXAM POINT

The limiting factor for better retrieval is the same as for grammar and style checkers: the absence of complete grammatical and semantic analysis of the text. This causes information noise, the delivery of irrelevant documents, as well as the missing of relevant ones.

Summary

References to words and word combinations help authors write correct, flexible, and idiomatic text, and this knowledge is a genuine part of linguistic competence. A collocation is a combination of words commonly used together and cannot be built by word-by-word translation, which is why each language needs its own dictionary; CrossLexica is such a dictionary of word combinations, while WordNet and EuroWordNet add deeper semantic relations through synsets and links like hyponymy, hyperonymy, meronymy, and holonymy. An information retrieval system indexes documents by keywords, supports logical and "m of n" queries, enriches queries and stored keywords through a thesaurus, and ranks results by relevance. Recall and precision are the two main measures of an IRS and trade off against each other. The main limit on accurate retrieval is the lack of complete grammatical and semantic analysis of the text, the same barrier that constrains grammar and style checkers.

Topical Summarization

In many settings a program must decide, without human help, what a document is mainly about. That single answer supports several practical tasks: filing documents under their main subject, delivering documents on a chosen subject to interested users, indexing documents automatically in a retrieval system, and helping people orient themselves quickly in a large collection. This section treats that task as a restricted form of summarization and studies two contrasting systems that solve it, the knowledge-based Classifier and the statistical TextAnalyst.

What topical summarization is

DEFINITION

Topical summarization is the automatic task of determining the main topics a document mentions, rather than restating its ideas, so that a short list of topics stands in for the whole text.

The task can be seen as a special kind of summarization. In ordinary summarization by contents, a program tries to convey the main ideas of a text in a shorter form. Topical summarization is narrower: it reports only the topics mentioned in the document, which is why it carries the word "topical". The goal is to answer one question, namely what the document is about, and the output is a set of topics or labels rather than a shortened retelling of the argument. This is appropriate whenever the subject of a text matters more than its exact wording.

Why it is useful

Knowing the topic of a document supports many downstream tasks. A system can classify documents by their main subject, deliver documents on a chosen subject to the users who want them, index documents automatically in an information retrieval system, and help a reader orient quickly within a large set of documents. In modern practice the same idea drives news feed categorisation, spam filtering, and automatic search indexing.

```
graph TD
  A[Document] --> B[Topic finding]
  B --> C[Classify by subject]
  B --> D[Route to interested users]
  B --> E[Index for retrieval]
  B --> F[Orient in a large set]
```

Topical versus full summarization

Although both are forms of summarization, they answer different questions and produce different output.

Aspect	Full summarization	Topical summarization
What is conveyed	The main ideas of the text	Only the topics mentioned
Output form	A shorter piece of running text	A list of topics or key words
Question answered	What does it say	What is it about

EXAM POINT

Both are forms of summarization, but topical summarization reports the topics mentioned, whereas full summarization by contents restates the main ideas. Do not treat them as the same task.

The Classifier system

DEFINITION

Classifier (a variant of Clasitex) is a knowledge-based program from the Center of Computing Research, National Polytechnic Institute, Mexico City, that determines the main topics of a document using a large topic thesaurus.

Classifier relies on two kinds of linguistic information. First, it uses morphology to reduce each word to its dictionary form. Second, it uses a large thesaurus that maps each form to a position in a predefined hierarchy of topics. The program then counts topic occurrences and selects the most weighted topic as the main topic of the document.

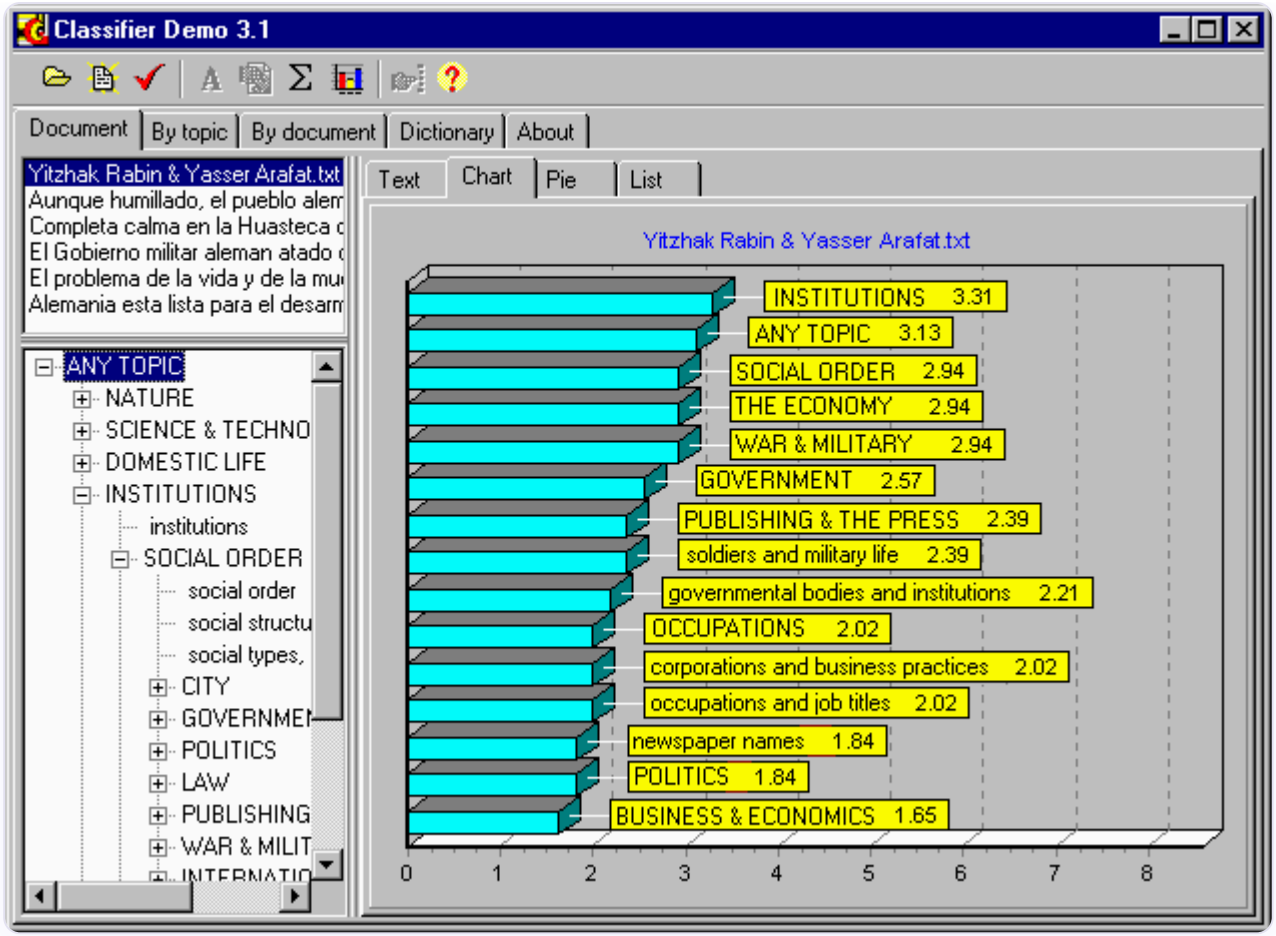


Figure: Bolshakov & Gelbukh, Computational Linguistics (2004)

Step one: normalise word forms

DEFINITION

Morphological normalization reduces every word found in the text to its standard, dictionary form so that all inflected variants count as one word.

Different inflected forms of a word should count together rather than separately. In English, *sentences* and *sentence* both reduce to *sentence*, *runs*, *running*, and *ran* all reduce to *run*, and *proposed* reduces to *propose*. Without this step, occurrences would scatter across many surface forms and no single form would look frequent enough to signal a topic.

Step two: map words to topics

The thesaurus gives each standard form a position in a hierarchy of topics, and topics nest inside one another so that a narrow topic sits within a broader one. Each word therefore contributes not to a single topic but to a whole chain of topics. For example, the word *sentence* belongs to *linguistics*, which belongs to *social sciences*, which in turn belongs to *science*. The program counts how often each topic occurs across the document.

```
graph TD
  A[sentence] --> B[linguistics]
  B --> C[social sciences]
  C --> D[science]
```

KEY POINT

Topics carry weights of importance, so the main topic is the one with the greatest total weight in the document, not simply the most frequent word.

A known limitation of Classifier

The gathered statistics are not fully correct, because some real references to a topic are never counted.

Two sources of error illustrate where applied linguistics could improve the method. First, pronouns are skipped, because it is not obvious which object a personal pronoun such as *it* refers to. Consider the pair *There is a book. It is very interesting.* The book is mentioned twice, once by name and once by the pronoun *it*, but a counter that matches only the string "book" records a single mention. Second, many languages go further than English and allow a subject to be left out altogether, so a whole sentence can be about the book without containing any word for it at all. Each such sentence is really one more mention of that noun, which Classifier does not count, so the statistics for that noun are silently too low.

The TextAnalyst system

DEFINITION

TextAnalyst is a system from MicroSystems, Russia, that finds the main topics of a document and the relationships between words, without a topic dictionary.

Unlike Classifier, TextAnalyst is not dictionary-based. It keeps only a small list of stop words, which are prepositions, articles, and similar function words that carry little meaning and should not be processed as meaningful words. The system reveals relationships between words: two words are considered related if they co-occur closely enough in the text, for example within the same sentence. From these relationships the program builds a network of connections between words.

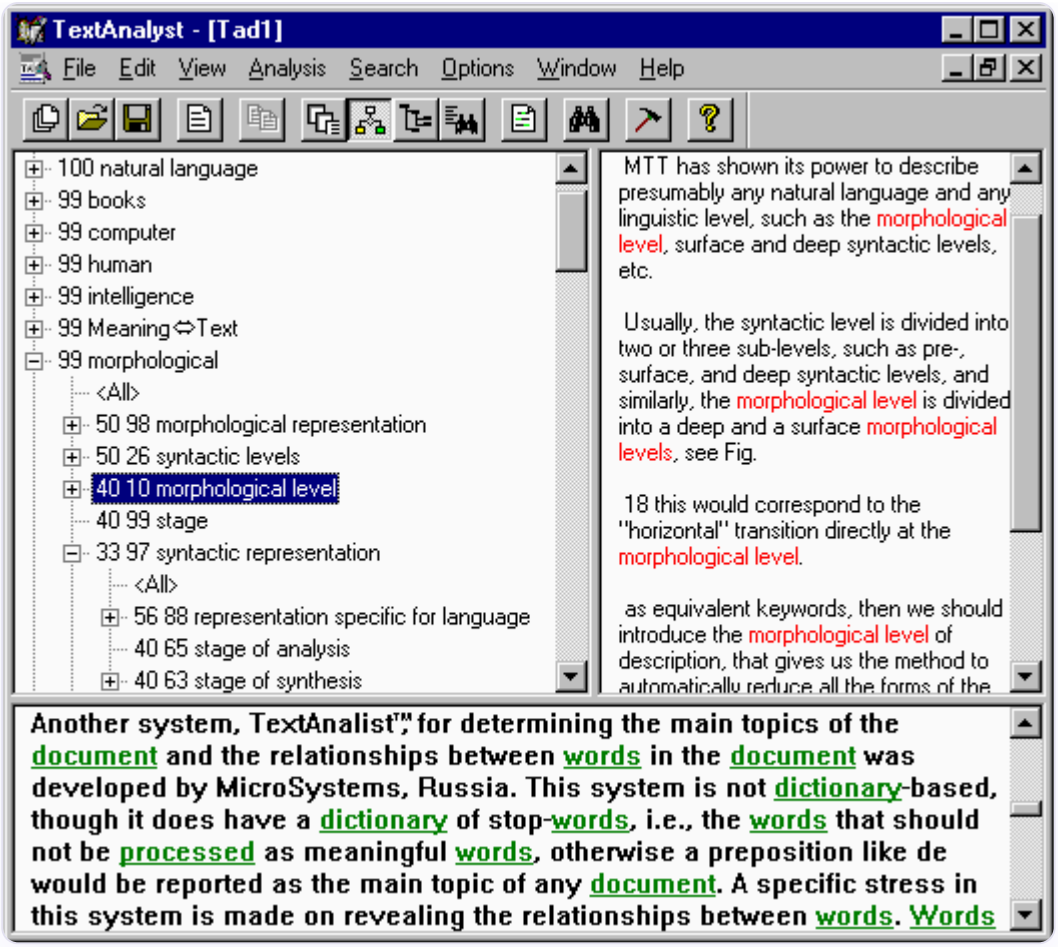


Figure: Bolshakov & Gelbukh, Computational Linguistics (2004)

The figure shows the most important words TextAnalyst found in an early draft of this textbook, together with the network of their relationships. There is no thesaurus behind this result, only the text itself.

How TextAnalyst weights words

As in Classifier, the importance or weight of a word begins from its frequency. The system then uses the network of relationships between words to mutually increase their weights: words that are closely related to many important words are themselves treated as important. In this way importance spreads across the network rather than being read from a fixed dictionary.

```
graph LR
  A[Count word frequency] --> B[Link co-occurring words]
  B --> C[Build relationship network]
  C --> D[Related words boost each other]
  D --> E[Ranked important words]
```

The technology is based on a special type of dynamic neural network algorithm, which spreads importance across the network of linked words.

What the important words are used for

The ranked list of important words drives two different outputs. In text compression, the system removes the sentences or paragraphs that contain the fewest important words, continuing until the text reaches a size threshold selected by the user. In hypertext building, the system constructs mutual references among the most important words and links important words to others to which they are supposedly related.

The same ranked list of important words drives two different outputs: a shortened document and a browsable web of links.

Knowledge-based versus statistical

The two systems represent two broad strategies for topic finding.

Aspect	Classifier	TextAnalyst
Basis	Large topic thesaurus	Word co-occurrence
Type	Knowledge-based	Statistical, network-based
Needs a dictionary	Yes	No, only stop words
Main output	Topics in a hierarchy	Important words and their links

EXAM POINT

Classifier is knowledge-based because it depends on a large dictionary; TextAnalyst is not, because it derives everything from the text using a neural network algorithm.

Summary

Topical summarization finds what a document is about, reporting topics rather than ideas, and it supports classification, routing, indexing, and orientation within large collections. Classifier normalises word forms, maps them through a topic thesaurus, and selects the topic with the greatest total weight, which makes it a knowledge-based program. TextAnalyst instead links co-occurring words into a network and lets importance spread across it using a dynamic neural network algorithm, so it needs no dictionary beyond a short list of stop words. The two systems together illustrate the contrast between knowledge-based and statistical approaches to the same task.

Automatic Translation

Automatic, or machine, translation is the conversion of text from one natural language into another by a computer program. It is one of the oldest goals of computational linguistics and, at the same time, one of the hardest to achieve well. The task is important because the volume of business and scientific text is growing rapidly, many countries publish heavily in their own languages, and international contacts create a constant need to translate legal contracts, technical documentation, instructions, advertisements, and the everyday texts read by millions of people. This section defines machine translation, traces the two historical approaches and why each fell short, assesses the quality of present-day software and the genres where it is usable, and works through the two central difficulties: word sense disambiguation and the recovery of implicit information.

Learning outcomes

By the end of this topic you should be able to define machine translation and state why it is important, contrast the word-by-word approach with the interlingua approach and explain why both proved inadequate, assess the quality of current translation software and name the genres where it is and is not usable, explain word sense disambiguation and implicit-information recovery as the two central difficulties, and read a commercial translator example and describe the errors it makes.

What machine translation is

DEFINITION

Machine translation (MT) is the automatic conversion of text in a source language into equivalent text in a target language by a computer program, without human translation of the content.

Machine translation is a product of computational linguistics, an application that people actually buy and use, not merely a research idea. Its importance is driven by the rapid growth of business and scientific text worldwide. Many countries are very productive in scientific and business domains and publish numerous books and articles in their own languages. With the growth of international contacts and collaboration, the need to translate legal contracts, technical documentation, instructions, advertisements, and other everyday texts has become a matter of vital importance, and the volume grows faster than human translators can keep up.

```
graph TD
  A[Global text volume grows] --> B[More languages in use]
  B --> C[Translation demand rises]
  C --> D[Human translators are scarce]
  D --> E[Need for automatic translation]
```

Figure: why demand for automatic translation keeps rising.

The first hope: word by word

DEFINITION

Word-by-word translation replaces each source word with a target-language equivalent taken from a bilingual dictionary, with no analysis of grammar or meaning.

The first programs for automatic translation were developed more than 40 years ago, in the 1950s and 1960s. At first there was a hope that texts could be translated word by word, so that the only problem would be to build a dictionary of pairs of words: a word in one language and its equivalent in the other. That hope died just after the very first experiments, because a word is not the unit of meaning across languages: word order, ambiguity, and grammar cannot be handled by dictionary lookup alone.

The ambitious goal: interlingua

DEFINITION

An **interlingua** is a universal, language-independent representation of meaning; the program analyses the source text into this representation, then generates the target text from it.

After word-by-word translation failed, an ambitious goal was formulated: to create programs that could deeply understand the meaning of an arbitrary text in the source language, record that meaning in some universal intermediate language, and then reformulate it in the target language with the greatest possible accuracy. It was supposed that neither manual pre-editing of the source text nor manual post-editing of the target text would be necessary. This goal proved tremendously difficult to achieve and has still not been satisfactorily accomplished in any but the narrowest special cases. It is the opposite extreme from word-by-word translation: full understanding rather than none.

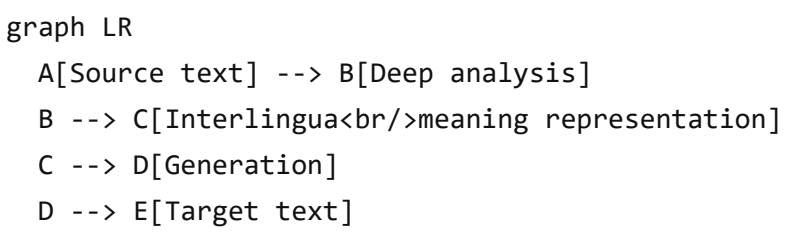


Figure: the interlingua ideal: analyse to meaning, then generate from meaning.

How good is it today

KEY POINT

Even for the largest systems built by the best scientists, translation quality is usually clearly below that of a competent human translator.

At present there is a great deal of translation software, ranging from very large international projects developed by several institutes or corporations in cooperation, down to simple automatic dictionaries, and from laboratory experiments to commercial products. The quality of commercial translation is still rather low. A commercial translator can let people quite unfamiliar with the original language understand the main idea of a document, and it can assist manual translation. However, post-editing the results to a quality sufficient for publication often takes more time than translating by hand a person who knows both languages well.

Where it works and where it fails

Commercial translators are quite good for texts of very specific, narrow genres, such as weather reports, because such genres have a small, predictable vocabulary and fixed phrasing. They are also acceptable for legal contracts, at least for their formal parts, but the paragraphs specifying the very subject of the contract may be somewhat distorted, because that is where the real meaning lives. General, meaning-rich prose comes out full of errors and must be re-edited.

Genre	Suitability for MT
Weather reports	Good, very narrow and formulaic
Legal contracts, formal parts	Acceptable
Legal contracts, subject clauses	May be distorted
General prose	Full of errors, needs re-editing

EXAM POINT

MT quality depends heavily on genre. Narrow, repetitive text such as weather reports translates well; open, meaning-rich prose does not. Be ready to name a good genre and a bad genre.

A classic mistranslation

The spirit is willing, but the flesh is weak.

To show what kind of errors a translation program can make, the book recalls a well-known example from one of the earliest systems in the 1960s. It translated the Bible verse *The spirit is willing, but the flesh is weak* (Matt. 26:41) into Russian and then back into English, and the result was *The vodka is strong, but the meat is rotten*. Similar examples still appear in modern translation systems today. The failure is one of sense selection: "spirit" and "flesh" each have several senses, and the machine chose the wrong one in each direction.

A commercial translator

The book demonstrates the popular commercial package Power Translator by Globalink, described as one of the best on the market at the time, translating English into Spanish. The header of an English document, "Plans", was translated as a verb, "makes plans", whereas the correct output is the noun. This is a part-of-speech error: the same string can be a noun or a verb, and out of context the system guessed wrongly. A second sentence was rendered as "the paper of France in the war", whereas the correct translation is "the role of France in the war". This is a word-sense error: the source word carries both the sense "sheet of paper" and the sense "role", and the system chose the wrong one. These two errors preview the two central difficulties discussed next.



Figure: Bolshakov & Gelbukh, Computational Linguistics (2004)

Direction matters

The quality of translation by a given program is not the same in the two directions, for example from English to Spanish and from Spanish to English. Because automatic analysis of text is usually a more difficult task than generation of text, translation from a language that is studied and described better generally has higher quality than translation into that language. Thus, improving Spanish grammars and dictionaries can raise the quality of translation from Spanish into English, because parsing the input is the bottleneck.

```
graph TD
  A[Translation quality] --> B[Depends on direction]
  B --> C[Analysis is harder<br/>than generation]
  C --> D[From a well-described<br/>language is easier]
  A --> E[Improved by statistics]
  A --> F[Improved by richer<br/>grammars and dictionaries]
```

Figure: why translation quality depends on the direction of translation.

Difficulty one: word sense disambiguation

DEFINITION

Word sense disambiguation is the task of choosing, for an ambiguous source word, which of its several possible target-language translations fits the current context.

In any bilingual dictionary, many source words list dozens of target-language translations. For the simple English word "bank", a bilingual dictionary must distinguish the edge of a river, a financial institution, a row of switches, a tilt of an aircraft, and the act of relying on something, and each of these is a different word in the target language. Which one should the program choose in a given case? This problem has proven extremely difficult to solve. Deep linguistic analysis of the text is necessary to make the correct choice, based on the meaning of the surrounding words, the text as a whole, and perhaps some extralinguistic information beyond the text itself.

KEY POINT

Correct sense selection requires deep linguistic analysis, not a dictionary lookup.

Difficulty two: recovering implicit information

DEFINITION

Implicit-information recovery is making explicit in the target text information that the source text left implied, such as gender, number, or formality that the target grammar demands.

A second, often more difficult, problem is restoring information that the source text contains implicitly but which must be expressed explicitly in the target text. The target language forces a choice the source never marked, so the machine must infer it from context or knowledge, and the necessary clue may be far away or absent. For example, the English sentence *They are beautiful* says nothing about the gender of the things referred to, yet many target languages require the adjective to carry a gender. The machine must look back through the preceding text to find what *they* refers to, and that clue may be several sentences away, or absent altogether. The examples below all require the same kind of inference.

Source	Ambiguity to resolve
A subjectless source sentence	English must supply he, she, it, or this
English "computer shop"	The target must decide whether "computer" is singular or plural
English "they are beautiful"	Many target languages force a gender on the adjective
English "as you wish"	Many target languages force a choice of singular or plural, formal or familiar
English "my cousin"	Many target languages force a gender, and some the side of the family

Again, deep linguistic analysis and knowledge, rather than simple word-by-word translation, are needed to solve such problems.

Where research leads

Great effort is devoted worldwide to improving translation quality. As an example of successful research, the book cites the Translation group of the Information Science Institute (ISI) at the University of Southern California, whose work is based on statistical techniques for lexical ambiguity resolution. Another successful team, headed by Yu. Apresian in Russia, works within the Meaning to Text model. Today's statistical and neural MT systems descend from the statistical line mentioned here.

Summary

Machine translation automatically converts text between languages and is in high and growing demand because of the rising volume of business, scientific, and legal text. The first hope, word-by-word translation from a bilingual dictionary, failed at once; the more ambitious interlingua goal of fully automatic, high-quality translation is still unmet outside the narrowest special cases. Current software conveys the gist of a document but usually falls short of human quality and needs post-editing, and it works well only for narrow genres such as weather reports while failing on open, meaning-rich prose. Quality also differs by direction, because analysis is harder than generation. The two central difficulties are word sense disambiguation, choosing the right target word for an ambiguous source word, and recovering implicit information, making explicit the gender, number, or formality that the target grammar demands but the source left unmarked. Both require deep linguistic analysis rather than simple lookup, and research such as the statistical work at ISI and the Meaning to Text model at Apresian's group aims to close the gap.

Natural Language Interface and Extraction of Factual Data

This section covers two related products of computational linguistics that both work at the boundary between natural language and structured data. The first is the natural language interface, which lets a human ask an application system questions in ordinary words and receive answers in kind. The second is the extraction of factual data from texts, which lets a machine read free-flowing prose and record the facts it contains in a structured database. One lets a person query a machine in plain language, the other lets a machine read plain language and populate a database, but both rest on the same idea of bridging natural language and formal data. By the end you should be able to define a natural language interface and state the task it performs, describe the structure of a system fronted by such an interface, explain why interface sublanguages are limited and why this makes analysis simpler than translation, explain the dialogue problem including elliptical questions and analysis through synthesis, and define information extraction and describe how factual data is drawn from on-line texts to fill a factographic database.

What a natural language interface does

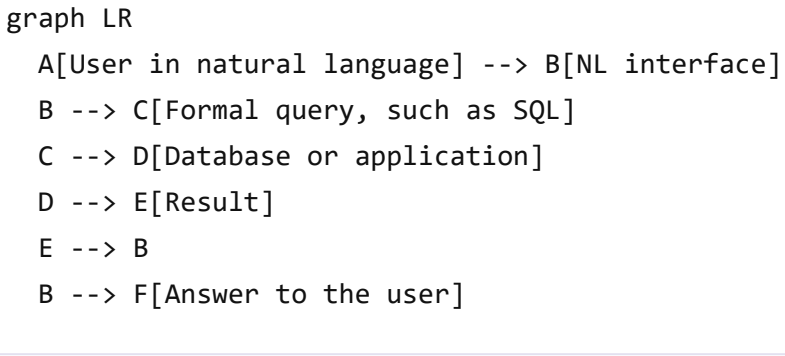
DEFINITION

A **natural language interface** to a database is a system whose task is to understand questions entered by a user in natural language and to provide answers, usually in natural language and sometimes as formatted output.

The queries typically concern facts about data held in a database. The user asks in ordinary words, and the system replies either in words or as a table or other formatted output. The aim is to remove the need to learn a formal query language, so that someone who wants to know, for example, which printers a store holds can simply ask rather than compose a formal statement. Two directions of work are involved: understanding the incoming question, and forming a useful answer.

Structure of the system

The interface sits between the user and an existing application system. It converts a natural language question into a formal query, the application answers, and the interface then presents the result back to the user. The database itself does not change: the interface is essentially a translator wrapped around a normal database.



Why the sublanguage is limited

DEFINITION

A **sublanguage** is the restricted subset of a natural language, with a limited vocabulary and grammar, that arises because each database covers a specialised subject.

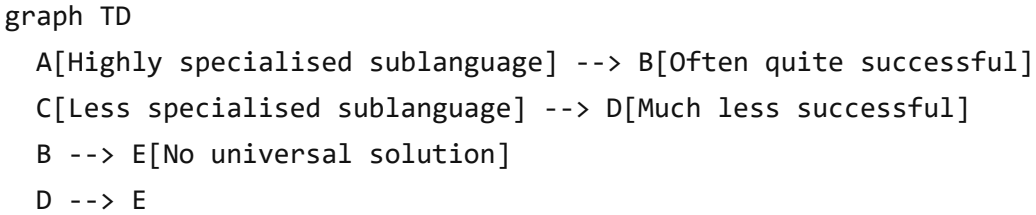
Because each database is specialised to some degree, the words used in queries and the queries themselves are narrow in scope. A store-inventory interface only ever hears about printers, prices, and stock, so the vocabulary it must handle stays small. A narrow sublanguage makes grammatical and semantic analysis much simpler, which means the task is far easier than open tasks such as machine translation, where the system must cope with unrestricted text.

KEY POINT

The narrower the subject, the smaller the vocabulary, and the more tractable the analysis.

How successful are these systems

Some systems understand a very specialised sublanguage quite well. Others, with less specialised sublanguages, are much less successful. There is no universal solution so far, and most interfaces are built ad hoc for each specific system. Success tracks how tightly the domain is bounded, and a general-purpose natural language interface remains an open research goal.



> The authors were not aware of any commercial interface working with a truly unlimited natural language.

Competition from formal query languages

Popular database systems ship with a formal query language such as SQL. SQL is not too hard to learn, which reduces the pressure for a natural language interface, since a trained user already has a working way to query the data. Even so, building such an interface stays attractive to research teams worldwide. The real prize lies in spoken access rather than typed queries.

```
graph TD
  A[Formal query language, SQL] --> B[Not too hard to learn]
  B --> C[Reduced need for NL interface]
  D[Research interest] --> E[NL interface still pursued]
```

KEY POINT

Speech recognition would make an interface especially useful, allowing queries or commands over a telephone line.

Which branches of linguistics matter

Interfaces make different demands on the branches of linguistics. Morphology and syntax are less demanding here, because the words are few and simple, so surface analysis is easy. The deeper branches are very demanding, because understanding intent in context is hard.

Branch	Demand for interface work
Morphology	Less demanding
Syntax	Less demanding
Semantics	Very demanding
Pragmatics	Very demanding
Theory of discourse	Very demanding

EXAM POINT

Interfaces are light on morphology and syntax but heavy on the deeper branches: semantics, pragmatics, and theory of discourse.

Dialogue, not monologue

DEFINITION

The specific problem of interface systems is that they work not with a narrative or monologue but with a **dialogue**, a set of short, incomplete, interleaving remarks.

A running conversation carries meaning across turns, so later remarks depend on earlier ones for their sense. The system must track the exchange as a whole, not just parse one sentence in isolation. In dialogue a two-word reply can carry a full question that only makes sense given what came before.

The elliptical question problem

Consider a short exchange. The user asks, "Are there wide high-resolution matrix printers in the store?" The system answers, "No, there are no such printers in the store." The user then says, "And narrow?" This last remark must be re-formulated using the earlier turn, because on its own it carries no complete meaning.

```
graph TD
  A[Full first question] --> B[System answer]
  B --> C["Elliptical reply: And narrow?"]
  C --> D[Re-formulate from context]
  D --> E[Are there narrow high-resolution matrix printers?]
```

KEY POINT

"And narrow?" means "Are there narrow high-resolution matrix printers in the store?" only when the previous turn is carried over.

Analysis through synthesis

DEFINITION

Analysis through synthesis is understanding an elliptical question by building a model of the user's goals, knowledge, and interests, then guessing what the system itself would ask at this point of the dialogue and in what words.

The system models the user's current goals and knowledge, imagines being the user and formulating the next question, and then matches that against the sparse input. The reversal is the key point: to understand, the machine generates. It asks in effect, "what would I say here?" and uses the answer to fill in the gaps of the elliptical remark. Detailed linguistic analysis is needed to reach the intended meaning.

Extraction of factual data from texts

DEFINITION

Information extraction, or extraction of factual data from texts, is the automatic generation of elements of a factographic database, such as fields or parameters, based on on-line texts.

The source is often current news from the Internet or from an information agency. Parameters of interest include the demand for a product by region, the prices of specific products, events involving a particular person or company, or opinions on an issue or a political party. The output populates a structured, factographic database. This is the mirror image of the interface: instead of answering questions from a database, the system builds the database by reading text.

Why it is worth doing

Officials in business and politics are usually too busy to read and comprehend all the relevant news in the time available. Hiring news summarizers and readers is expensive, and even then important relationships between facts may be lost, because each summarizer typically has very limited knowledge of the subject matter. A fully effective automatic system can not only extract the relevant facts much faster but also combine them, classify them, and investigate their interrelationships across many stories.

```
graph LR
  A[On-line news texts] --> B[Extraction system]
  B --> C[Extract relevant facts]
  C --> D[Combine and classify]
  D --> E[Factographic database]
```

Laboratory systems exist for business use, for example one that explores news on the Dow Jones index, investments, and company merger and acquisition projects. Because the task is so difficult, mainly large commercial corporations can afford the research.

Summary

A natural language interface understands user questions in ordinary language and returns answers, usually in language and sometimes as formatted output. It sits between the user and an existing application, turning questions into formal queries such as SQL while leaving the database itself unchanged. Limited sublanguages make analysis easier than translation, but interfaces remain demanding on semantics, pragmatics, and theory of discourse. Dialogue brings elliptical questions, which are resolved by analysis through synthesis over a model of the user. Information extraction works in the opposite direction, reading on-line texts to fill a factographic database with facts that can then be combined and classified.

Text Generation, Language Understanding, and Related Systems

The final products of computational linguistics group into three families that sit at very different levels of maturity. Text generation runs the translation pipeline in reverse: instead of understanding a text it builds one, starting from pictures or from a formal specification. Language understanding is the most general and the most difficult goal of the whole field, because a system that truly understands a text can, in principle, perform every other language task. Related systems such as optical character recognition, handwriting recognition, and speech recognition are not usually counted as computational linguistics proper, yet they cannot reach high accuracy without leaning on linguistic knowledge. This section defines each task, sets out the knowledge it needs, and marks the distinctions that separate a shallow solution from a deep one. It closes the review of applied systems and connects to modern natural language processing, where large language models now attempt generation and understanding together.

Text generation

DEFINITION

Text generation is the task of deriving a coherent natural language text automatically from pictures or from a formal specification rather than from another text.

Text generation is a comparatively new field that arose about ten years before the source book was written, and useful applications appeared only gradually. The defining feature is the input: the system does not start from a text in another language, as translation does, but from non-textual or highly formal data. From that input it must produce prose that a human can read as a natural, connected whole.

Where text generation is used

Two settings show the range of the task. Multimedia systems often need a text-generating subsystem that illustrates a picture with a textual explanation, producing coherent text from the features of the picture. A second and important application is the generation of formal specifications in text form from formal technical drawings.

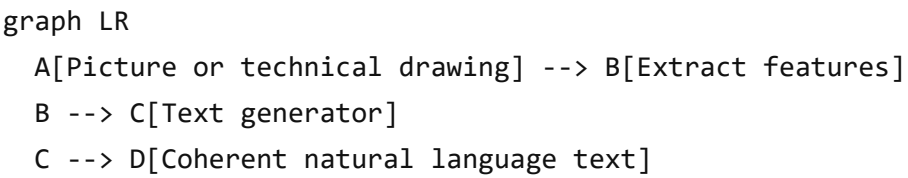


Figure: text generation builds prose from non-textual or formal input.

A concrete example is the compilation of a patent formula for a new device. Such a document is often many pages long, and for a human it is boring, time consuming, and error prone. The task suits a machine far better than a person, because the machine does not tire and applies the same rules consistently.

Multilingual text generation

DEFINITION

A **multilingual text generating system** produces the required description or set of instructions independently in each target language, working from the technical drawings, specifications, or a formal source language rather than translating one finished text.

In many cases a new device needs descriptions and instructions in several languages, or in as many as possible. The obvious route, writing the text once and translating it, gives poor results, because the quality of automatic translation of a manually compiled text is often very low for the reasons discussed under translation. Better results come from generating the required text in each language independently, starting from the drawings and specifications or from a text in a specific formal language similar to a programming language. Each language then gets prose built directly for it rather than a translation of prose built for another.

KEY POINT

Generating each language independently from a formal source avoids the errors that come from translating a single finished text, so multilingual generation usually beats generate-once-then-translate.

How much linguistic knowledge it needs

Text generating systems carry, in general, about half of the linguistic problems of a translation system. They avoid the analysis of a source text, but they inherit every problem tied to the grammar and lexicon of the target language, because they must produce correct output in that language. This is still a vast body of linguistic information, which is currently available in adequate detail only for a few very narrow subject areas. That limitation explains why practical generation systems tend to be confined to restricted domains such as technical manuals and patents.

```
graph TD
  A[Technical drawings and formal spec] --> B[Generate in English]
  A --> C[Generate in French]
  A --> D[Generate in Spanish]
```

Figure: generating each language independently from one formal source, rather than translating a finished text.

Systems of language understanding

DEFINITION

Natural language understanding is the transformation of a text into an internal, meaning-preserving representation that a purposeful automaton can use directly for reasoning.

Natural language understanding systems are the most general and most complex systems that involve natural language processing. They are universal in the sense that a single understanding system can perform nearly all the tasks of other language-related systems: grammar and style checking, information retrieval, automatic translation, natural language interface, extraction of factual data, text generation, and so on. Automatic translation, for instance, can be built as an understanding system that first understands the source text and then generates a precise description of the learned information in the target language. For this reason, creating a text understanding system is the most challenging task for the joint efforts of computational linguistics and artificial intelligence.

Two cooperating modules

An understanding system is not only a linguistic program. The natural language processing module is only one part; most of the logical reasoning and understanding proper are concentrated in a separate reasoning module. The two are closely interrelated and must work in tight cooperation.

DEFINITION

The **reasoning subsystem** is the part of an understanding system that performs logical reasoning over the internal representation, and depending on the whole system it may be called a knowledge-based reasoning engine, a problem solver, or an expert system.

The linguistic subsystem is usually bi-directional: it can both analyze the input texts or queries and generate another text as the answer. It transforms a human utterance into an internal, or semantic, representation that the reasoning subsystem can process, the reasoning subsystem produces a response in its own internal format, and the linguistic subsystem then transforms that response back into a textual answer. Whatever name the reasoning part carries, its role in the whole system is always very important.

```
graph LR
  A[Human utterance or query] --> B[Linguistic subsystem: analyze]
  B --> C[Internal semantic representation]
  C --> D[Reasoning subsystem]
  D --> E[Response in internal format]
  E --> F[Linguistic subsystem: generate]
  F --> G[Textual answer]
```

Figure: the bi-directional loop between the linguistic and reasoning subsystems.

The Turing test and the state of the goal

Half a century before the source book, Alan Turing suggested that the principal test of intelligence for a computer would be its ability to conduct an intelligent dialogue: making reasonable decisions, giving advice, or simply presenting relevant information during the conversation.

DEFINITION

The **Turing test**, as Turing framed it, treats a machine as intelligent if it can hold an intelligent dialogue, reasoning and advising as a human would during conversation.

This great goal had not been achieved when the book was written, although artificial intelligence and computational linguistics had pursued it for about thirty years. To build what can honestly be called a true language understanding system, the full power of linguistic science and the full range of linguistic data and knowledge are necessary. Modern large language models have narrowed the gap on dialogue, but genuine understanding, in the strict sense of reliable meaning-preserving reasoning, remains an open problem.

EXAM POINT

Language understanding is the most general and least developed application: one understanding system can subsume translation, retrieval, and generation. Be ready to describe its two modules, the bi-directional linguistic subsystem, and why the Turing test frames the goal.

Related systems

Some applications are not usually classed as computational linguistics proper, yet they rely heavily on linguistic methods. Two of them, both forms of pattern recognition, are worth examining: character recognition and speech recognition.

Optical character recognition

DEFINITION

Optical character recognition (OCR) recognizes graphemes, that is, letters, numbers, and punctuation marks, in a point-by-point image of printed text and converts them to the corresponding character codes.

The graphemes may appear in any font, typeface, or size, and the paper background may carry stray dots and spots. Image recognition proper is beyond the competence of computational linguistics, but the task cannot be solved by image recognition alone. A person reads a degraded image easily because they understand the meaning of the words; without that meaning, an isolated letter cannot be identified reliably. Presented with a single ambiguous mark, one cannot decide whether it is an a, an s, or a g, yet the same mark is read at once inside a full word. The lesson is direct: linguistic information is required even to recognize the shapes correctly.

KEY POINT

OCR cannot reach high accuracy from image analysis alone, because letters that are ambiguous in isolation become readable only when the surrounding words supply meaning.

Even after a clean image is recognized, peculiar errors still appear in the textual output, and they can be fixed by operations like those of a spell checker. A specialized spell checker for OCR should know the most frequent recognition errors: the lower-case l resembles the digit 1, the letter n is often read as the pair ii, and m as iii or rn, while the digraphs in, rn, and ni are often merged into m. Most such errors can be corrected without human intervention using linguistic knowledge, in the simplest case a dictionary of the words that exist in the language. Harder cases need deeper analysis; only full parsing of the surrounding sentence can decide whether a string recognized as modem stands for modem or for modern, since the pair rn is so often merged into m.

```
graph TD
  A[Point-by-point image of printed text] --> B[Image recognition]
  B --> C[Raw character codes with errors]
  C --> D[Specialized spell checker: known OCR errors]
  D --> E{Still ambiguous?}
  E -- No --> G[Corrected text]
  E -- Yes --> F[Full parsing of the context sentence]
  F --> G
```

Figure: OCR needs a linguistic correction stage after image recognition.

Handwriting recognition

DEFINITION

Handwriting recognition converts into character codes the texts written by hand, whether on paper, on the surface of a special device, or directly with a mouse on the screen.

Handwriting recognition is a more challenging task than the recognition of printed text, but the main kinds of problem and the methods of solution are nearly the same, at least in their linguistic aspect. The same reliance on a dictionary and, where needed, on parsing carries over from printed OCR.

Speech recognition

DEFINITION

Speech recognition recognizes the specific sounds in the flow of human speech and converts them into the character codes of the corresponding letters.

The recognition itself belongs both to pattern recognition and to phonology, the science on the border of linguistics, acoustics, and physiology that studies the sounds used in speech. The difficulties are very similar to those of OCR: mutilated patterns, fused patterns, disjoint parts of a pattern, lost parts, and noise superimposed on the pattern. Because speech is even harder to segment than print, it yields an even larger number of incorrectly recognized letters, which makes the application of linguistic methods even more important than in OCR.

Aspect	Optical character recognition	Speech recognition
Input	Point-by-point image of printed text	Flow of spoken sounds
Neighbouring science	Image recognition	Phonology
Typical distortions	Spots, ambiguous letter shapes	Noise, fused or lost sounds
Error rate	Lower	Higher
Role of linguistic knowledge	Important	Even more important

EXAM POINT

For related systems, be ready to explain why OCR and speech recognition both need linguistic knowledge, list the shared distortions, and state that speech recognition has the higher error rate and the greater dependence on linguistic methods.

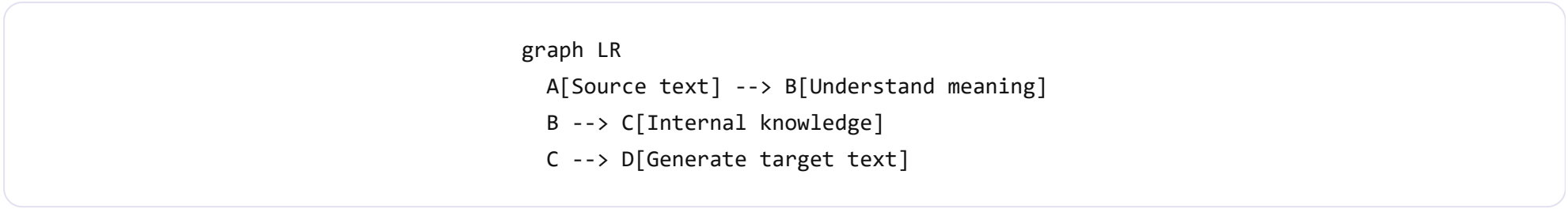


Figure: translation routed through understanding rather than through surface correspondence.

Why deep linguistic knowledge is unavoidable

A short review of applied linguistic systems shows a clear pattern. Only very simple tasks, such as hyphenation or simple spell checking, can be solved on a modest linguistic basis. Every other system needs relatively deep linguistic knowledge: dictionaries, morphological and syntactic analyzers, and in some cases deep semantic knowledge and reasoning. Even spell checking must use very deep analysis to reach an accuracy approaching 100 percent. Above all, most language processing tasks can be viewed as special cases of the general task of language understanding, which remains one of the ultimate goals of both computational linguistics and artificial intelligence.

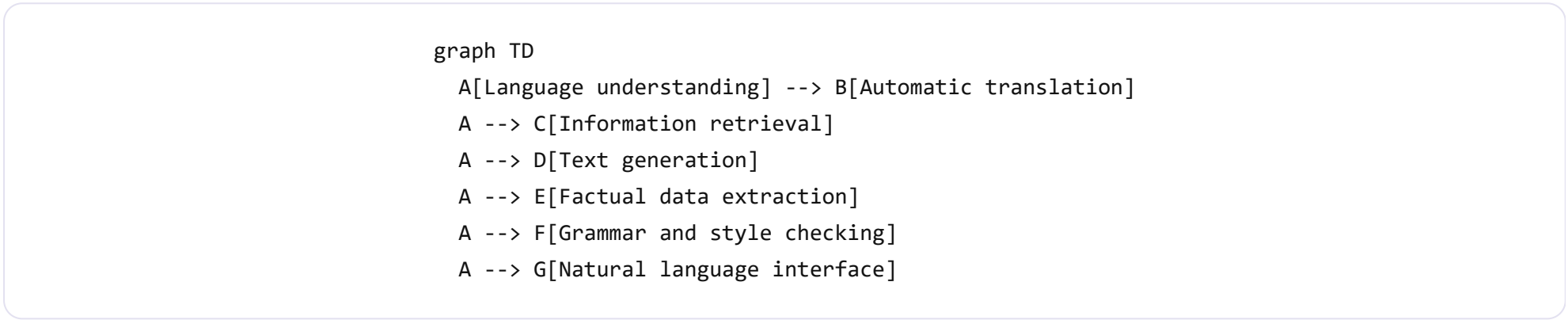


Figure: most language tasks are special cases of general language understanding.

Summary

Text generation derives coherent prose from pictures or formal specifications, with patents and multilingual technical documentation as leading uses; generating each language independently from a formal source beats translating one finished text, and the task still carries about half the linguistic burden of translation, limited to narrow domains. Language understanding is the most general and most complex application: one understanding system can subsume translation, retrieval, generation, and more, working through a bi-directional linguistic subsystem paired with a reasoning subsystem, with the Turing test as the framing goal that remains unreached in the strict sense. Related systems, optical character recognition, handwriting recognition, and speech recognition, are pattern-recognition tasks that nonetheless depend on dictionaries and parsing, with speech recognition the hardest and the most reliant on linguistic methods. Overall, only the simplest tasks survive on shallow knowledge, while everything else, and especially anything aiming near perfect accuracy, reduces to some measure of language understanding.